

MEDHAVA

The seven stages

Your seven, in your order, with your numbering. Each one says who does it — because several of these cannot be done by any AI, and a plan that leaves that unsaid is a plan that stops at the first one.

At a glance

#	Stage	Who	Where it stands
1	Map the Umbrella Domain and DNS Hierarchy	YOU, and only you	nothing registered yet
2	Design the BOM Data Engine and Schema	MOSTLY DONE	TESTED
3	Construct the Shared API and Single Sign-On (SSO)	PARTLY EXISTS, and the gap is real	TESTED
4	Develop the Core Web Application	IN PROGRESS, and this is where most of the remaining work is	TESTED
5	Build Companion Mobile Apps	YOU, AND NOT STARTED	SPECIFIED
6	Implement Offline-First Caching and Sync	YOU, AND NOT STARTED	SPECIFIED
7	Deploy Infrastructure and Monitoring Pipelines	YOU, AND NOT STARTED — nothing has ever been deployed anywhere	SPECIFIED

1 · Map the Umbrella Domain and DNS Hierarchy

Decide which name serves what before anything is built on them. One name for the public site, one for the application behind sign-in, one for internal tools, and mail pointed somewhere else entirely. "Umbrella" is the

useful part: one domain at the top, everything else a name underneath it, so a second business or a second product later is another name rather than another domain and another certificate.

Who does this. YOU, and only you. This needs the login to wherever you bought the domain, which is a credential no AI should ever hold and this one never will.

Before you start. You own the domain. Nothing else is needed.

What gets done

- Write down the four names and what each serves — the table in DEPLOYMENT.md §4
- Leave them unpointed until the machine exists, then point them all at it
- Point mail records at your mailbox provider, never at the application machine

You know it worked when: Each name resolves to the machine, checked from a connection that is not your own.

Detail: `DEPLOYMENT.md`

2 · Design the BOM Data Engine and Schema

The tables every other stage reads and writes, and the rule that one company can never see another company's rows. A bill of materials — what a finished thing is made of, and therefore what has to be bought and consumed to make one — is part of this and is specified rather than built: module 08 Manufacturing is on the queue, not in the product.

Who does this. MOSTLY DONE. The schema and the isolation exist and are proven against a real Postgres. The bill of materials itself is not built.

Before you start. The database exists and the application connects to it as a restricted role.

What exists today, read from the requirements registry:

Capability	Status	What that means
<code>CAP-SCHEMA</code> The production schema	TESTED	a test drives it, it passes, and the run is on record
<code>CAP-ISOLATION</code> Tenant isolation in the database	TESTED	a test drives it, it passes, and the run is on record

What gets done

- Load core/schema.postgres.sql into your database
- Create the application role as DEPLOYMENT.md §6a sets out — not the superuser, not the table owner, or every isolation policy is inert
- Run `node core/tests/live.test.js` against your real database

You know it worked when: That test passes against your database. It is the single check that proves isolation holds, and it only proves it because the role is restricted.

3 · Construct the Shared API and Single Sign-On (SSO)

One way in for everything — the web app, the mobile app, an automation — so that signing in once is enough and every one of them is subject to the same permissions. "Shared" is the whole point: the mobile app in stage 5 must not get its own private door with its own rules, because two doors mean two sets of rules and one of them will be wrong.

Who does this. PARTLY EXISTS, and the gap is real. Sign-in, sessions and company switching work. A public API — versioned, with keys, rate limits and webhooks — does not exist and is registered as not started. Single sign-on against an outside identity provider does not exist either.

Before you start. Stage 2 is done. The database is real and the role is restricted.

What exists today, read from the requirements registry:

Capability	Status	What that means
<code>CAP-SHELL</code> Sign-in, session and the web shell	TESTED	a test drives it, it passes, and the run is on record
<code>CAP-DEVPLATFORM</code> Developer platform — public API, webhooks, SDKs	NOT STARTED	nothing exists

What gets done

- Run what exists: `npm start`, then `npm run medhava` to see its tests pass
- Decide whether sign-on is your own accounts or an outside provider BEFORE the mobile app — changing it afterwards means reissuing every credential
- Build the public API surface as its own slice, with the same gates as everything else

You know it worked when: One account signs in to the web app and the mobile app, and revoking it stops both.

4 · Develop the Core Web Application

The application people actually use. The design names 22 modules and 113 apps; what runs on the real database today is a much smaller number, and the registry states exactly which. Everything else is written down, which is not the same as built.

Who does this. IN PROGRESS, and this is where most of the remaining work is. `BUILD_QUEUE.md` is the ordered list of what to do next and why each item is in that position.

Before you start. Stage 3 — there is one way in, and it works.

What exists today, read from the requirements registry:

Capability	Status	What that means
CAP-SHELL Sign-in, session and the web shell	TESTED	a test drives it, it passes, and the run is on record

What gets done

- Open BUILD_QUEUE.md and take the first task that is not done
- Build one vertical slice at a time — screen, rules, database, test — never a layer at a time across many features
- Prove each rule fails before it passes; a check never seen to fail proves nothing
- Record the run through tools/evidence.js so the claim survives the conversation

You know it worked when: A person in the business does a real day of work in it and stops using the spreadsheet they used before.

Detail: [BUILD_QUEUE.md](#)

5 · Build Companion Mobile Apps

"Companion" is the important word. Not the whole system on a phone — the two or three things that genuinely happen away from a desk: marking attendance, reporting what was produced, approving something. A phone app that tries to be the whole application is how mobile projects die.

Who does this. YOU, AND NOT STARTED. There is no mobile code in this project at all. It also needs accounts I cannot hold — Apple and Google developer accounts, both paid and both requiring identity verification that takes days.

Before you start. Stages 3 and 4. There is a shared way in, and there is something worth opening.

What exists today, read from the requirements registry:

Capability	Status	What that means
CAP-MOBILE Mobile	SPECIFIED	written down, not standing up

What gets done

- Decide the two or three things that must work on a phone, and refuse the rest
- Start the developer accounts early — the verification is slow and blocks release
- Build against the shared API from stage 3, never against the database directly

You know it worked when: Somebody on the floor marks attendance on a phone and it appears in the web application without anybody re-entering it.

Detail: [MEDHAVA_PLAN_OF_ACTION.md](#)

6 · Implement Offline-First Caching and Sync

The phone keeps working where there is no signal and catches up later. The hard part is not the caching — it is deciding what happens when two people changed the same thing while both were offline. That decision is a business rule, not a technical detail, and it has to be made by someone who knows the business.

Who does this. YOU, AND NOT STARTED. Nothing here is built. Do not let it be added quietly to stage 5 as though it were a setting.

Before you start. Stage 5. There is an app to be offline in.

What exists today, read from the requirements registry:

Capability	Status	What that means
CAP-MOBILE Mobile	SPECIFIED	written down, not standing up

What gets done

- List every action that must work offline — it will be shorter than it first seems
- Decide, per action, who wins when two offline changes collide, and write it down
- Build it as its own slice with its own tests, including the collision cases

You know it worked when: A phone in aeroplane mode records a day's work; when it reconnects, every record it made is in the system, and where somebody else changed the same record while it was offline the result is the one your written rule says it should be.

Detail: [MEDHAVA_ARCHITECT.md](#)

7 · Deploy Infrastructure and Monitoring Pipelines

Getting it onto a machine the business can reach, keeping it there, and knowing before your customers do when it stops. Backups belong here, and a backup nobody has restored is not a backup.

Who does this. YOU, AND NOT STARTED — nothing has ever been deployed anywhere. It needs a machine, a domain and credentials, none of which exist yet and none of which I can obtain. DEPLOYMENT.md is the runbook and has never been followed by anybody, so expect it to be wrong in small ways the first time and fix it as you go.

Before you start. Something worth deploying. In practice this runs alongside stage 4 rather than after it — deploy early and often, with little in it, rather than once with everything.

What exists today, read from the requirements registry:

Capability	Status	What that means
CAP-DEPLOY Deployment to a production environment	NOT STARTED	nothing exists
CAP-MONITOR Monitoring, alerting and health checks	SPECIFIED	written down, not standing up
CAP-PUBLISH The static site published at a public URL	BLOCKED	cannot proceed — the row says what is blocking it

What gets done

- Follow DEPLOYMENT.md from §1 to §11, in order
- Deploy something small before deploying something important
- Restore a backup once, deliberately, before you need to
- Set the health check and point an uptime checker at it

You know it worked when: You deliberately stop the application and an alert reaches you before anyone in the business notices.

Detail: [DEPLOYMENT.md](#)

Stages 1, 5, 6 and 7 need you, not a tool

Worth saying once more in one place, because it is the thing most likely to stall:

- **The domain** needs your registrar login.
- **The mobile apps** need Apple and Google developer accounts, both paid, both requiring identity verification that takes days. Start them before you need them.
- **Offline sync** needs a business decision — who wins when two people changed the same thing while both were offline — that only somebody who knows the business can make.
- **Deployment** needs a machine and credentials.

None of these is difficult. All of them are slow if started late.

Every technical word above, in plain language

16 words. Every technical term this document uses, in plain language, with an everyday comparison. Nothing here assumes you already know any of them.

platform

One piece of software that many separate businesses use at the same time, each seeing only its own information.

Ek badi building jisme bahut saare offices hain. Building ek hai, par har office ki chaabi alag — koi kisi aur ke office mein nahin ghus sakta.

tenant

One business using the platform. Its people, its data and its settings are its own.

Us building mein ek office. Aapka office, aapka saamaan, aapka taala.

module

One area of work in the system — sales, purchase, staff, accounts. Each is a set of screens that belong together.

Dukaan ke alag-alag counters. Ek counter bikri ka, ek kharidi ka, ek hisaab-kitaab ka.

database

Where all the information is kept, arranged so any of it can be found instantly and nothing gets lost.

Ek badi almari jisme har cheez apne fix khaane mein rakhi hai — dhoondhne ke liye poori almari palatni nahin padti.

table

One kind of information inside the database — all your customers in one, all your orders in another.

Almari ka ek khaana. Ek khaane mein sirf customers, doosre mein sirf orders.

row

One single record — one customer, one order, one payment.

Register mein ek line. Ek line matlab ek entry.

schema

The written plan of what information the system keeps and how the pieces connect.

Makaan ka naksha. Deewar uthane se pehle kaagaz pe tay hota hai kaunsa kamra kahaan hai.

backup

A copy of everything, kept somewhere else, so a mistake or a failure does not lose your work.

Zaroori kaagzaat ki photocopy, doosri jagah rakhi hui. Asli jal jaaye toh bhi kaam nahin rukta.

API

The agreed way two pieces of software talk to each other, so one can ask the other for something and get a predictable answer.

Waiter. Aap kitchen mein nahin jaate — waiter ko order dete ho, wahi khaana le aata hai. Waiter badal jaaye toh bhi order dene ka tarika wahi rehta hai.

queue

A waiting line for work that does not have to finish this second — sending a hundred messages, building a big report.

Darzi ki dukaan ka parchi system. Kaam parchi pe likh ke lag gaya line mein; customer khada intezaar nahin karta.

environment

A separate running copy of the system — one for trying things, one that customers actually use.

Rehearsal aur asli show. Practice alag jagah, taaki galti sabke saamne na ho.

deployment

Putting a new version of the software in place so people start using it.

Nayi dukaan kholna ya purani ko naya roop dena — jab tak shutter nahin uthta, customer ko farq nahin padta.

uptime

How much of the time the system is actually working and reachable.

Dukaan mahine mein kitne din khuli rahi. Band rahi toh customer wapas chala gaya.

provider

A company whose service the system uses — for messages, for payments, for artificial intelligence, for delivery.

Supplier. Ek supplier maal na de toh doosre se le lo — kaam nahin rukna chahiye.

role

What a person is allowed to see and do — a manager sees more than a counter staff member.

Chaabi ka guccha. Manager ke paas zyada chaabiyaan, staff ke paas kam.

permission

One specific thing a role is allowed to do, like approving a discount or viewing salaries.

Guchhe ki ek chaabi. Ek chaabi ek darwaza.

